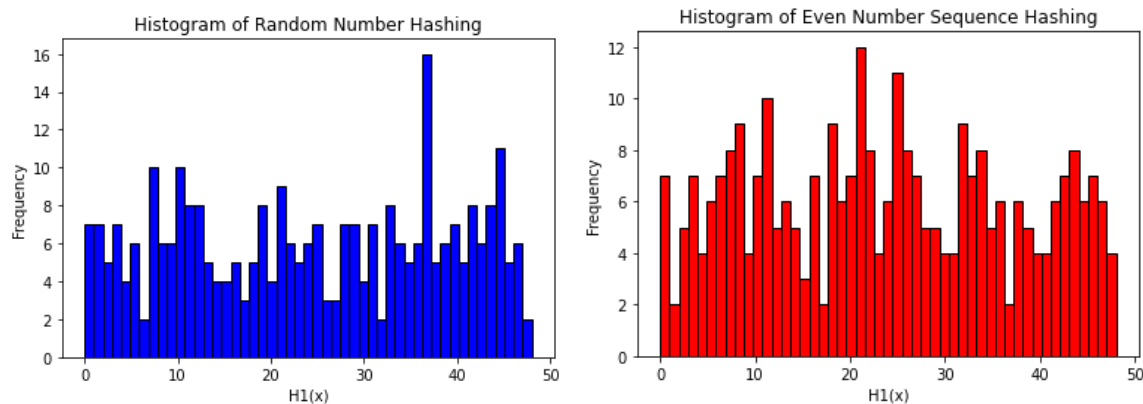


1. Hash Functions Overview

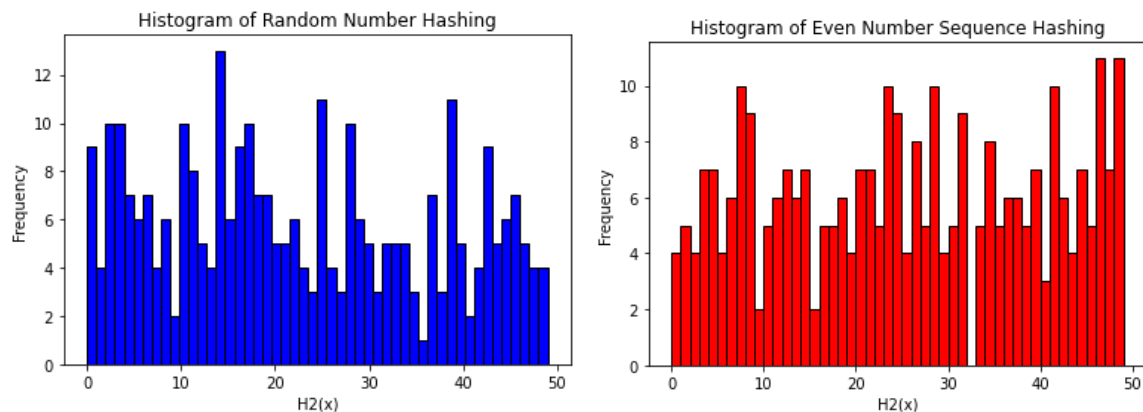
I decided to run a mini experiment that previews the sampling and hashing procedure we will run in later cells. Let N be a very large number with $m = 50$ and $k = 6$. This means that we are mapping random numbers generated in the set U to the set of integers from 0 to 49; the same is done when mapping the first $2m$ even numbers up to m ; this is seen in the blue and red histograms respectively.

Let's start with the first Hash function:



These histograms demonstrate that for the first hashing scheme, which uses a random heavy approach, a random subset of U produces slightly more extremities in max/min loads compared to consecutive even numbers. However, this is not enough evidence to decide a verdict comparing the two hashing schemes.

Now with the second Hash function:



These histograms demonstrate that for a intricate hashing scheme with randomness and direct modulo operation, a balanced subset like the even number sequence is able to produce a more well-rounded distribution; the blue bins reach higher extremes in max load while having not much more instances of min load compared to red.

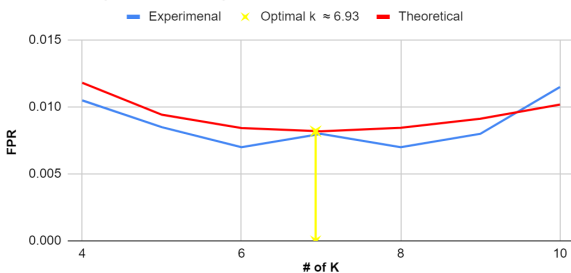
In conclusion, data choice may matter in that correlated, distinct data plays a better role in hashing schemes that use direct mapping (modulo) and not as random-heavy. Yet, we can see that there is virtually no significant difference between the actual two hashing functions. This will be further discussed later in the report!

2. Implementing the Bloom Filter

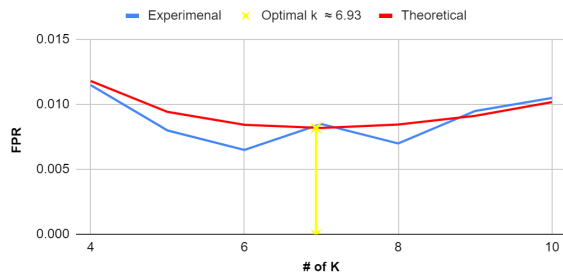
I decided to instantiate the Bloom Filter as a class so building, performing operations, and rebuilding is convenient. There are three class methods: the required `add(x)` and `contains(x)`, but also `hash(x)`. This is so I can conveniently return the hashing index for each hashing, `add`, and `contains` of a certain `x` (useful for the previous section when returning the hashing mapped values then plotting). I opted to code the two hashing schemes in the same general operation- this required the hashing type argument to be specified and passed in & outside the bloom filter class. Because I specify the N value later on, I set the prime number variable to follow that value I end up choosing, but needs manual change whenever N is changed. Furthermore, I chose to take care of producing the k seeds in a helper “False Positive Rate” calc function.

3. Analyze False Positive Rate (FPR) for different values of k

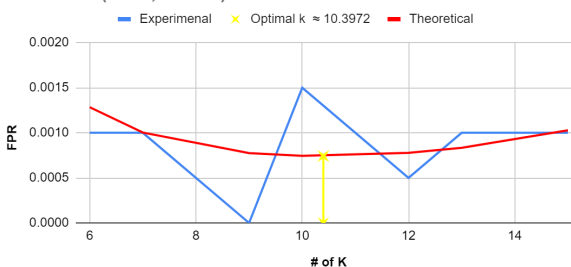
FPR v.s K ($c=10$, Hash1)



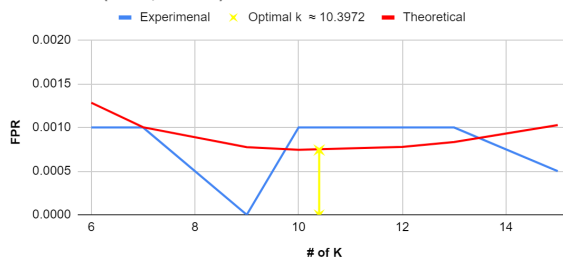
FPR v.s K ($c=10$, Hash2)



FPR v.s K ($c=15$, Hash1)



FPR v.s K ($c=15$, Hash2)



I used python to generate the data arrays (7 values) for experimental FPRs (median format), following the given procedure for trials in Part 3 of the project instructions. Using google sheets, I was able to make 4 plots: 2 c -values, 10 and 15, with 2 line graphs each for the two Hash schemes.

I observed that the performance of the two schemes too be very similar. Lower c-values allow the relationship between FPR and #k to follow the theoretical graph trend where the optimal k was calculated using $k = c \ln 2$ and $FPR = (1 - e^{-k/c})^k$.

As such, we can find the optimal k-value in the legend for the yellow lines. The first c=10 experiment was successful and supports the theoretical result. On the other hand, as we increase to the c=15 experiment, we can infer that the results are more unpredictable.

Here, for both Hash types, we find the optimal k to be significantly less than that of the theoretical. Yet, the false positive rate clearly follows a different trend and is capable of making jumps to higher and very low values of FPR. I was able to produce FPR values of 0; certainly, a higher amount of queries and c-values should be explored when evaluating the efficacy of my study.

In conclusion, the two hash schemes behave very similar in terms of their false positive rates; experimental settings may let users expect to observe a slightly less number of optimal k hash functions than that of the theoretical.